

MS SQL Server Distributed Partitioned Views Part 2

By Don Schlichting

In this second article, the use of Distributed Partitioned Views for Insert, Update, and Delete statements will be explored.

Introduction

In Part1, the basics of Distributed Partitioned Views, Federated Databases, and Horizontal Partitioning were reviewed. A sample Linked Server, a partitioned table and a view using the UNION operator were created. In this second part, a new sample demonstrating DML statements (INSERT, UPDATE, and DELETE), will be developed.

Links

We will start by creating a test environment. The examples will be created using two machines, each running SQL Server on Windows 2000. Although there will only be two machines in our examples, the same rules would apply for three or more.

Begin by creating reciprocal Linked Servers, from Server1 to Server2, and from Server2 to Server1. Log in to Server1 as sa and execute the following code.

```
USE master
GO
```

```
EXEC sp_addlinkedserver
    @server = 'server2',
    @srvproduct = 'SQLServer OLEDB Provider',
    @provider = 'SQLOLEDB',
    @datasrc = 'InfoNet'
```

```
GO
```

```
EXEC sp_addlinkedsrvlogin 'server2', 'false', 'sa', 'sa', 'secret'
```

```
GO
```

```
SELECT *
FROM server2.pubs.dbo.authors
```

All the rows from the authors table should have been returned. Log in to Server2 as sa and execute the following code:

```
USE master
```

GO

```
EXEC sp_addlinkedserver
    @server = 'server1',
    @srvproduct = 'SQLServer OLEDB Provider',
    @provider = 'SQLOLEDB',
    @datasrc = 'dons13'
```

GO

```
EXEC sp_addlinkedsrvlogin 'server1', 'false', 'sa', 'sa', 'secret'
GO
```

```
SELECT *
FROM server1.pubs.dbo.authors
```

For an explanation of the above code, see Part1.

Tables

With the Linked Servers created, the next step is to create sample tables. Imagine an orders table containing gigabits of past sales history, and reporting performance is getting sluggish. We are going to split this large table in half. Server1 will house sales with order numbers less than 1,000. Orders greater than 1,000 will be stored on Server2.

From Server1 execute these statements from master to create the test table:

```
CREATE DATABASE test
GO
```

```
USE test
GO
```

```
CREATE TABLE [dbo].[OrdersUnder] (
    [ord_nbr] [int] NOT NULL ,
    [ord_date] [datetime] NOT NULL ,
    [cust_id] [int] NOT NULL,
    [amount] [money] NOT NULL
) ON [PRIMARY]
GO
```

```
ALTER TABLE [dbo].[OrdersUnder] WITH CHECK ADD
    CONSTRAINT [PK_ord_nbr] PRIMARY KEY CLUSTERED
    (
        [ord_nbr]
    ) ON [PRIMARY]
GO
```

```
ALTER TABLE [dbo].[OrdersUnder] WITH CHECK ADD
    CONSTRAINT [CHK_Under] CHECK ([ord_nbr] <= 1000)
GO
```

From Server2, the code is almost identical. Only the name and constraint changes:

```
CREATE DATABASE test
GO
```

```
USE test
GO
```

```
CREATE TABLE [dbo].[OrdersOver] (
    [ord_nbr] [int] NOT NULL ,
    [ord_date] [datetime] NOT NULL ,
    [cust_id] [int] NOT NULL,
    [amount] [money] NOT NULL
) ON [PRIMARY]
GO
```

```
ALTER TABLE [dbo].[OrdersOver] WITH CHECK ADD
    CONSTRAINT [PK_ord_nbr] PRIMARY KEY CLUSTERED
        [ord_nbr]
ON [PRIMARY]
GO
```

```
ALTER TABLE [dbo].[OrdersOver] WITH CHECK ADD
    CONSTRAINT [CHK_Over] CHECK ([ord_nbr] > 1000)
GO
```

There are several items to note. First, a Check Constraint is required. The Query Optimizer uses the Check to determine on which server the data should be located. The check must allow data to be directed to one, and only server.

Second, a Primary Key is required for DML. Selects do not require a key. However, for Inserts, Updates and Deletes, the key must be present. In addition, the Key cannot be created as an Auto Identity field. Using one will produce the error:

```
Server: Msg 4433, Level 16, State 4, Line 1
Cannot INSERT into partitioned view 'ViewName'
    because table '[TableName]' has an IDENTITY constraint.
```

MS SQL Server Distributed Partitioned Views Part 2 - Page 2

By Don Schlichting

Views

This view is a simple select pulling data from both servers into one result set. From Server1, create a view with the following statement:

```
CREATE VIEW AllOrders
AS
SELECT *
FROM OrdersUnder
UNION ALL
SELECT *
```

```
FROM server2.test.dbo.OrdersOver
```

For Server2, again the code is almost identical:

```
CREATE VIEW AllOrders
AS
SELECT *
FROM OrdersOver
UNION ALL
SELECT *
FROM server1.test.dbo.OrdersUnder
```

There are a couple of rules for DML Views. For a view to be updatable, all the columns included in the primary key must be returned. Any columns not included in the view, must allow Nulls.

Test the view by running a simple select:

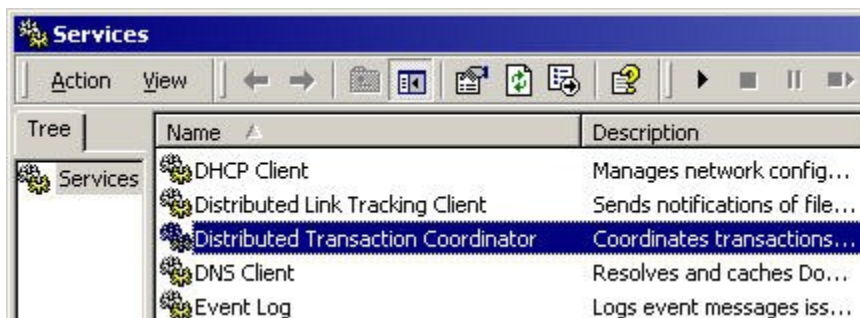
```
SELECT *
FROM AllOrders
```

An empty record set should be returned. Before starting any DML statements, there are a couple more items to include.

Distributed Transaction Coordinator

Before beginning the examples, we need to start the Distributed Transaction Coordinator. The DTC manages the committing of transactions when there are several different data sources involved. For Windows 2000, service pack 1 is required.

Open the services MMC, locate and start the Distributed Transaction Coordinator using the default settings.



Lazy Schema Validation

Although not a requirement, setting lazy schema validation will increase query performance. Schema validation is the act of checking the remote schema to validate its metadata. By setting the validation to lazy, SQL will not validate the remote metadata against our query until execution. If there has been a schema change on the remote, our query will error out.

In our case, we know the remote table is valid, with the same structure as our local table. We will pickup a gain by not checking remote schema.

```
use master
GO
sp_serveroption 'LocalServerName', 'lazy schema validation', true
GO
sp_serveroption 'server2', 'lazy schema validation', true
GO
```

Collation Compatible

If we assure SQL that the sort and character set are the same between the local and remote servers, then the remote can participate in comparisons. Otherwise, all the data will come across the wire, and be returned to the local server. All comparisons will then be done locally, degrading performance. This can be avoided by setting the collation compatible option to true. Again, this is not a requirement for distributed queries, but an optimization option.

```
use master
GO
sp_serveroption 'LocalServerName', 'collation compatible', true
GO
sp_serveroption 'server2', 'collation compatible', true
GO
```

DML

Let's begin by inserting data into the new empty table. The following statement will insert one record:

```
use test
GO
SET XACT_ABORT ON
GO
INSERT INTO AllOrders
    (ord_nbr, ord_date, cust_id, amount)
VALUES
    (1001, '01/01/1993', 5, 50.25)
```

The insert itself is the same transact SQL used regularly. XACT_ABORT is required for data modification statements. When set ON, any run time error will cause the entire transaction to roll back.

Selecting from our view will verify the record has been written successfully:

SELECT*				
FROM AllOrders				
	ord_nbr	ord_date	cust_id	amount
1	1001	1993-01-01 00:00:00.000	5	50.2500

Deleting records involves the same steps:

```
SET XACT_ABORT ON
GO
DELETE
FROM AllOrders
WHERE cust_id = 5
```

The newly inserted record is now deleted. Notice the WHERE clause does not need to reference our primary key or check constraint column.

Odds and Ends

The view can reference the linked table by four part naming (as our example has done), by using the OPEN ROWSET function, or the OPENDATASOURCE function.

Insert and update are not allowed on tables containing a timestamp column.

For a complete list of view rules, see BOL "partitioned views."

Although we now have two tables, on two different servers, acting as one, there is no automatic way to backup, or restore them, as one unit.

Conclusion

Federated Databases along with Distributed Partitioned Views can increase performance on very large tables. Care and planning will be required, especially for DML operations. Nevertheless, the performance gains can be worth the extra administration.